

LcsHelper v3 — Algorithm Overview

ULTRASAT Low Cadence Survey (LCS) scheduler

LcsHelper v3 is a simplified, retry-based scheduler for the ULTRASAT LCS. It fixes the survey anchor to `First_day` (no anchor search), uses extinction-only categorization, repairs scheduling failures by shuffling fields between sets, places up to 4 SetD fields with an integrated `correct_Inds`-equivalent repair, and finally builds the per-day per-slot schedule matrix. It is a deliberate step back from the joint-optimization complexity of v2 in favor of a simpler structure.

1. Pipeline overview

The scheduler runs in two phases. **Preparation** computes visibility tables and does an initial extinction-only categorization. **Scheduling** places A/B/C with retry, then SetD (with integrated repair), then builds the per-day matrix.

Step	Method	Purpose
1	<code>calc_vis_matrix</code>	Per-day visibility per field from Sun/Moon/Earth exclusion
2	<code>calc_cont_vis_windows_v2</code>	Continuous-visibility runs per field (with 1d-gap variant)
3	<code>categorizeFields_v3</code>	Initial split into Sets A/B/C by extinction only
4	<code>categorize_then_schedule</code>	Orchestrator: try schedule, shuffle, retry, SetD, daily matrix

Steps 1-3 run in `prepTablesBeforeSchedule`. Step 4 runs in `categorize_then_schedule` (this replaces the v2 `buildSchedule`) and internally invokes `schedule_SetA_v3`, `schedule_SetC_v3`, `schedule_SetB_v3`, `schedule_SetD_v3`, and `calcDailySchedule`.

2. Key design choices (and what changed from v2)

Aspect	v2 behavior	v3 behavior
Anchor	Scored over 60+ candidates	Fixed at <code>First_day</code>
Nominal_windows	<code>First_day + 45*(0:7)</code>	Same
Full_windows	Derived from SetA anchor	Identical to Nominal
SetC_start_ind	Picked inside <code>schedule_SetC</code>	Tried 3 then 1 in orchestrator
Categorization	Extinction-only, then joint-matching attempts	Extinction-only; shuffles after failures
SetA placement	Constrained-first + group shifts	Single matchpairs; one group-shift rescue if needed
Failure handling	Hard error or partial	Shuffle one field, retry up to 10x
SetD	Hand-rolled placement + <code>correct_Inds</code> (buggy)	Pre-clean + per-rank Case A/B/C
<code>correct_Inds</code>	Separate step; multi-steal bug	Pre-clean (matchpairs); SetA preserved
Diagnostic prints	Always on (verbose by default)	Off by default; gated on <code>Obj.Verbose (=false)</code>

File size	~2200 lines	~1750 lines
-----------	-------------	-------------

v3 is the full A/B/C/D/Daily pipeline. SetD and the SetA-preserving repair (correct_Inds-equivalent logic) are integrated into schedule_SetD_v3. Plotting tools (plotSchedule, plotCatB) are inspection helpers, not part of the core scheduling.

3. Categorization (Step 3)

categorizeFields_v3 partitions the field pool using only extinction and visibility-window-length thresholds. No anchor awareness, no scheduling-feasibility look-ahead. This is the original simple rule, restored after v2's scoring experiments did not improve robustness.

3.1 Filter cascade

Filter	Condition
Low_ext_fields	$Av_ext \leq max_ext (=1.0)$ AND $max_window \geq Min_window (=45\ d)$
Long_low_ext	Low_ext_fields AND $max_window \geq Max_window_cut (=135\ d)$
Short_fields	Low_ext_fields \ Long_low_ext

If Low_ext_fields contains too few fields for $A+B+C+1 = 81$ candidates, the filter is relaxed to use the 1-day-gap-allowed visibility tables. Same fallback applies if Long_low_ext has fewer than 32 fields. The use1gap flag is recorded per field so the right visibility table is used downstream.

3.2 Set assignment

After sorting Long_low_ext by extinction ascending:

Set	Source	Count
SetB	Long, lowest-extinction 16	16
SetC	Long, next-lowest-extinction 16	16
SetA	Short_fields + remaining Long	~48
Long_leftover	Same as "remaining Long", kept for shuffling	~11

Long_leftover_fields is a v3-specific pool. It is initially identical to the "long fields beyond rank 32" subset of SetA, but kept as a separate table because the shuffle logic treats it as the candidate pool for swapping into SetB or SetC when scheduling fails.

4. Time-grid definitions

All scheduling uses a fixed 8-window time grid anchored at First_day:

```
Nominal_windows.start = First_day + Min_window * (0:7)
Nominal_windows.end   = Nominal_windows.start + Min_window - 1
Full_windows           = Nominal_windows (identical in v3)
```

With First_day = 1 and Min_window = 45, the eight Full_windows are [1,45], [46,90], [91,135], [136,180], [181,225], [226,270], [271,315], [316,360].

SetA grid divides these 8 windows into 6 SetA groups of 8 slots each:

```
SetA slot start = anchor_g + (s-1) * Min_window
where g = group (1..6), s = slot (1..8),
and anchor_g = First_day + (g-1) * 8 * Min_window for the un-shifted case
```

SetC super-windows are $3 \times \text{Min_window} = 135$ days long, starting at `Full_windows.start(SetC_start_ind)` and the day 135 days after:

```
sw1_start = Full_windows.start(sci)
sw2_start = sw1_start + 3*Min_window
sw_end    = sw_start + 3*Min_window - 1
```

SetB uses the same 8 `Full_windows` as `SetA`, distributed per a fixed `SetB_division` table that depends on `SetC_start_ind` (see Section 6).

5. The orchestrator: categorize_then_schedule

This is the new buildSchedule. It tries each SetC_start_ind option (3 first, then 1 as fallback) and at each option, attempts A/B/C scheduling up to MaxRetries = 10 times, shuffling one field between sets after each failure. After A/B/C success, it runs SetD (if RunSetD = true) and then builds the daily schedule matrix.

5.1 Control flow

```
for sci in [3, 1]:
    Obj.SetC_start_ind = sci
    categorizeFields_v3()          # reset to extinction-only
    for attempt = 1..MaxRetries:
        Obj.Schedule = empty
        [okA, unplaced_A] = schedule_SetA_v3()
        [okC, unplaced_C] = schedule_SetC_v3()
        [okB, unplaced_B] = schedule_SetB_v3()
        if all 3 succeeded:
            SUCCESS; break
        shuffled = shuffle_on_failure(unplaced_A, unplaced_C, unplaced_B)
        if not shuffled:
            no candidates left; break inner loop and try other sci
    if neither sci succeeded:
        warn; return
    if Args.RunSetD:
        schedule_SetD_v3()          # see Section 8
    try:
        calcDailySchedule()          # mandatory final step, Section 9
    catch ME:
        warn; Obj.Schedule preserved
```

The try/catch around calcDailySchedule is a safety net: if this final step fails on some edge case, Obj.Schedule is preserved so downstream consumers can still inspect the A/B/C/D placement.

5.2 Why retry-with-shuffle works

The initial categorization picks the 16 lowest-extinction long fields for SetB. Some of those fields may not have visibility geometry compatible with the rigid SetB_division demand histogram. The shuffle identifies such fields and swaps them for the next-best-extinction candidate from Long_leftover_fields. The replacement may still fail, in which case the next attempt tries another swap. After 10 attempts the algorithm gives up at this SetC_start_ind and tries the other.

6. The three A/B/C scheduling steps

6.1 schedule_SetA_v3

Places all 48 SetA fields. Two phases:

Phase 1. All 6 SetA groups anchored at First_day. Build a 48-slot bipartite graph (slots vs SetA fields) and solve with matchpairs. Edge cost is 0 if the field fits the slot, BIG (=1e6) otherwise. If matchpairs finds a perfect matching, done.

Phase 2. If Phase 1 leaves any slot unfilled, try shifting exactly one group by $\pm 1..30$ days. For each (g_shift, sh), rebuild slot_starts with group g_shift at the new anchor and rerun matchpairs. Take the first configuration that achieves full 48-field placement. If only partial improvements are found, keep the best partial result.

When Phase 2 commits any configuration (full or partial improvement) that involves a shifted group, the shifted group index is recorded in `Obj.SetA_shifted_group`. This is consumed by the SetD pre-clean and Case B logic (Section 8): SetA fields in the shifted group are ineligible to be moved because their visibility is tied to the shifted anchor, not the standard time grid.

6.2 `schedule_SetC_v3`

Places 16 SetC fields into two 135-day super-windows. `SetC_start_ind` (set by the orchestrator) selects which `Full_windows` index starts super-window 1.

Algorithm: `local_setC_place_constrained_first` sorts SetC fields by `max_window` ascending. Each field is placed into whichever feasible super-window has fewer fields so far (load-balanced). A field with zero feasibility against both super-windows is reported as unplaced.

6.3 `schedule_SetB_v3`

Places 16 SetB fields, each receiving a 135-day continuous observation split into three 45-day blocks (one high-cadence at 1 day, two low-cadence at 4 days) per the `SetB_division` table.

For `SetC_start_ind = 3`, the division table is:

```
W45_base    = [1 1 1 1 3 3 4 4 5 5 6 6 7 7 8 8]
W90_1_base  = [2 2 2 2 2 2 3 3 3 3 7 7 6 6 6 6]
W90_2_base  = [3 3 3 3 4 4 2 2 4 4 8 8 8 8 7 7]
```

For `SetC_start_ind = 1`, the values are mirrored via $9 - x$. The minimum of `W45`, `W90_1`, `W90_2` per row gives the `firstInd` for that row, and the histogram of `firstInds` gives the demand per `Full_window` index. For `sci = 3` the demand histogram is `{ind1: 6, ind4: 2, ind5: 4, ind6: 4}`.

For each SetB field, feasibility is computed at each demanded `firstInd` (continuous 135-day visibility starting at that `Full_windows` day). Then `matchpairs` assigns fields to division rows, with `demand_counts(k)` replicated identical slots per `firstInd k`. The matching maximizes cardinality (16/16 if feasible).

Row numbering convention (matches v1): For SetB rows, group encodes the `Full_windows ind` ($100 + W45_value$ for `B_45` rows, $200 + W90_value$ for `B_90` rows). `ind` is a within-group counter (1, 2, ..., `Nw`) rather than the `Full_windows ind` itself. This is essential for the cadence interleaving in `calcDailySchedule`: `mod(ind, 4)` spreads the `B_90` fields sharing one `Full_windows ind` across the four cadence offsets so that the daily slot at that `ind` is occupied by exactly one `B_90` field per day (instead of all of them piling on the same days).

7. Shuffle on failure

`shuffle_on_failure` applies one swap per call. It dispatches based on which scheduling step failed:

Failure type	Primary action	Fallback
SetB failed	Swap unplaced SetB field with lowest-extinction Long_leftover field	Swap with highest-extinction SetC field
SetC failed	Swap unplaced SetC field with lowest-extinction Long_leftover field	Swap with highest-extinction SetB field
SetA failed	Move a Long_leftover field from SetA into SetC	(only one path tried; if it fails the orchestrator gives up)

The primary action is "promote a fresh candidate from the leftover pool". When the leftover pool is empty (after many swaps), the fallback exchanges a field between SetB and SetC.

After any swap, the orchestrator wipes `Obj.Schedule` and tries scheduling again from scratch. Field tables (SetA/B/C and Long_leftover) are persistent across attempts within a sci.

8. SetD scheduling

SetD places up to 4 high-extinction fields, chosen in priority order from a WG5-supplied ranking. Each placement uses one ind of `Full_windows` and a single 45-day @ 1-day-cadence observation. SetD candidates typically have `Av_ext > max_ext` and are looked up directly from `Obj.AllSky` by field ID; visibility uses the strict `Cont_visibility_per_field` table (no 1d-gap variant).

The SetD pipeline has three logical phases: **slot accounting**, **pre-clean** (to clear any over-budget inds left by A/B/C), and a **per-rank placement loop**. The invariant is that every A/B/C field in `Obj.Schedule` remains scheduled throughout (possibly relocated to a different ind, but never removed).

8.1 Slot accounting

The helper `local_compute_slot_occupancy` computes the per-ind "filled slot count" from the existing A/B/C schedule. The accounting is cadence-aware: SetB_90 and SetC fields share daily slots in groups of 4 (one per cadence offset mod 4).

```
filled(k) = nA(k) + nB45(k) + (nB90(k) + nC(k)) / 4

where:
  nA(k)   = SetA  rows with ind == k
  nB45(k) = B_45  rows with group == 100 + k (Full_windows ind
          recovered from group, since B_45.ind is a counter)
  nB90(k) = B_90  rows with group == 200 + k (same convention)
  nC(k)   = SetC  rows whose super-window covers ind k
          (each SetC field covers 3 consecutive inds)
```

The four-day-cadence pool `nB90(k) + nC(k)` MUST be divisible by 4 in every ind — the four cadence offsets must all be filled to share one daily slot. If divisibility fails in any ind, a **warning is emitted unconditionally** (independent of `Verbose`) and SetD scheduling aborts. This signals a real bug in the A/B/C schedule that should be investigated before proceeding.

Outputs (with repetition):

- `inds_open(j)` — ind `k` appears (`Daily_LCS_slots - filled(k)`) times
- `inds_2move(j)` — ind `k` appears (`filled(k) - Daily_LCS_slots`) times when over-full

For the standard A/B/C placement under `sci=3`, the typical state is `inds_open = [1 2 2 2 5]` and `inds_2move = [3]` (ind 3 has 12 filled, ind 1 has 10, ind 2 has 8, ind 5 has 10, others exactly full). For `sci=1` the pattern is mirrored: `inds_open = [4 7 7 7 8]`, `inds_2move = [6]`.

8.2 Pre-clean: `clean_inds_before_setD`

Before placing any SetD field, the pre-clean ensures `inds_2move` is empty by moving SetA fields from over-budget inds to inds in `inds_open`. The matching is solved globally via `matchpairs` (rows = eligible SetA rows at over-budget inds, columns = `inds_open` entries; edge cost 0 when the SetA field is 45-day-feasible at the open ind, BIG otherwise).

Eligibility constraint: only SetA fields from non-shifted groups (`group ~= Obj.SetA_shifted_group`) can be moved. Fields in the shifted group have their visibility tied to the shifted anchor and cannot be relocated onto the standard time grid.

Each accepted match decrements `inds_2move` at the source ind and `inds_open` at the destination ind. The corresponding SetA row in `Obj.Schedule` gets its `ind`, `start`, `end`, and `group` updated. The new group is the lowest available value ≥ 7 at the target ind (so the first moved SetA at an ind gets group 7, the second gets 8, etc.). This keeps moved fields visually distinct from the original groups 1-6.

Indexing detail: the loop snapshots `inds_open` before consuming entries, because `pairs(r, 2)` indexes into the original column ordering — after deletions the running array shifts, so the snapshot is used to recover the destination ind value.

If pre-clean cannot fully clear `inds_2move` (no eligible SetA can bridge a remaining over-slot), it returns `ok = false` and SetD aborts.

8.3 Placement loop (Case A / B / C)

WG5 ranking (default): [79 12 48 28 16 88 55 32 213 26]. Iterate this list with a rank pointer; for each SetD slot, advance through ranks until a placement commits or the ranking exhausts. For each candidate:

- Compute feasible inds for this candidate (those where it has 45-day visibility starting at `Full_windows.start(k)`).
- **Case A — direct placement.** If any feasible ind is in `inds_open`, place the SetD field there and consume one entry from `inds_open`. Done.
- **Case B — bump a SetA field.** Otherwise, for each feasible ind `k_setD` NOT in `inds_open`, search SetA rows at `k_setD` (non-shifted group) for a field that has visibility at some `k_open` in `inds_open`. Among all such candidates across all `k_setD` options, pick the one with the FEWEST alternative inds (lowest "flex" — prefer constrained fields). Move that SetA to `k_open` (gaining a group number ≥ 7 per the renumbering rule) and place the SetD field at `k_setD`. `inds_open` loses `k_open`; `inds_2move` stays empty (one SetA out, one SetD in at the same ind).
- **Case C — skip.** If neither A nor B finds a placement, skip this rank and advance the pointer. The SetD slot will be filled by the next-ranked candidate that succeeds.

On successful return, `inds_2move` is empty (the invariant from pre-clean is preserved throughout). If the ranking exhausts before 4 fields are placed, a warning reports how many succeeded.

9. Daily schedule (calcDailySchedule)

The final step of `categorize_then_schedule` builds `Obj.Daily_schedule`: a matrix of size $(\text{Last_day} - \text{First_day} + 1) \times \text{Daily_LCS_slots}$ (typically 360×11) where entry (d, s) is the field ID observed in slot s on day d (NaN if no observation).

For each row in `Obj.Schedule`, the function fills the appropriate days according to the cadence rule for that category:

Category	Cadence	Daily placement rule
A	1 day	Every day in [start, end]
B_45	1 day	Every day in [start, end]
B_90	4 days	Days where $\text{mod}((d - \text{start} + 1), 4) == \text{mod}(\text{ind}, 4)$
C	4 days	Days where $\text{mod}((d - \text{start} + 1), 4) == \text{mod}(\text{ind}, 4)$
D	1 day	Every day in [start, end]

After populating, the function performs a within-day slot reordering: for each day, fields that are not visible across ALL slots (per `vis2d_day_field_ALL`) get relocated to a slot where they are visible (per `vis3d_slot_day_field`). If the constrained slot is in the lower half of the day, the earliest feasible slot is chosen; otherwise the latest. A guard handles the rare case where no feasible slot exists in the available range (the field is left at its current slot to avoid corruption).

`calcDailySchedule` is wrapped in a `try/catch` by the orchestrator so any unexpected error issues a warning but does not invalidate `Obj.Schedule`.

10. Diagnostic output (Verbose flag)

`v3` is silent by default. All diagnostic `fprintf` calls are gated on `Obj.Verbose`, which defaults to `false`. This keeps `StartDate` sweeps clean — a typical 100-day sweep produces no output unless a `warning(...)` is emitted (e.g. the divisibility check or the "no feasible schedule" warning).

To turn diagnostics on, pass `Verbose=true` at construction:

```
LCS = ultrasat.planner.LcsHelper_v3( ...
    'StartDate', '2029-02-01', ...
    'AllSkyTable', tbl, ...
    'Verbose', true);
LCS.prepTablesBeforeSchedule;
LCS.categorize_then_schedule;
```

When verbose, the orchestrator prints a header per `SetC_start_ind` attempt, a line per scheduling attempt, the per-step success/failure with unplaced fields, the specific shuffle applied, the pre-clean `SetA` moves, and every `SetD` Case A/B/C decision with the field IDs involved.

11. Plotting tools

Two inspection helpers are included for ad-hoc visualization. They are not called by the core pipeline.

Method	Purpose
<code>plotSchedule(Args)</code>	Timeline of all scheduled fields across categories A/B/C/D with category band markers

plotCatB(Args)	Per-field timeline for the 16 SetB fields (solid line = W45, dashed = W90 blocks)
----------------	---

Note on moved SetA in plotSchedule: SetA rows are plotted at $y = \text{group}$. The original 6 groups map to $y = 1..6$ (the SetA band $y < 8$). Fields moved by pre-clean or Case B receive groups ≥ 7 . Group 7 still falls in the SetA band; groups 8+ may visually drift into the SetB band. This is cosmetic only — the underlying schedule is correct.

12. Data structures

Property	Type	Role
vis3d_slot_day_field	logical [slots x days x fields]	Raw visibility per 15-min observing slot
vis_day_field	logical [days x fields]	Per-day visibility (any-slot or all-slots per Whole_daily_window)
Cont_visibilty_per_field	double [days x fields]	Length of the visibility run starting at each (day, field)
Cont_visibilty_per_field_1d_gap	double [days x fields]	Same but treating isolated 1-day gaps as still visible
Longest_window_per_field	double [1 x fields]	Maximum entry per column of Cont_visibilty (i.e. longest run length)
Longest_window_per_field_1dgap	double [1 x fields]	Same for the 1d-gap variant
SetA_fields, SetB_fields, SetC_fields	table	Field assignments; Field/Av_ext/use1gap/max_window/scheudled columns
Long_leftover_fields	table	Long-visibility fields not assigned to SetB or SetC (initially ~11)
SetD_ranked_fields	table	WG5 ranking with placement state (Field/Av_ext/scheudled/ind)
inds_open	array (with repetition)	After SetD: inds with free slots; ind k repeated per open count
inds_2move	array (with repetition)	After SetD: should be empty (cleared by pre-clean)
Nominal_windows	table (start, end)	Calendar grid: First_day + 45*(0:7)
Full_windows	table (start, end)	Working grid (identical to Nominal_windows in v3)
SetC_start_ind	double (1 or 3)	Which Full_windows index starts SetC super-window 1
SetA_shifted_group	double (0..6)	Group shifted by SetA Phase 2 (0 = none); SetD moves exclude this group
Schedule	table	Final output: category/group/ind/start/end/Field rows
Daily_schedule	double [days x slots]	Per-day per-slot matrix; entry = field ID or NaN

13. Removed from v2 (intentional simplification)

v3 removes machinery that did not survive scrutiny.

Removed item	Why
Anchor scoring loop (60+ candidates)	Anchor is now fixed at First_day; no scoring needed
setB_n_fits and setC_n_fits precomputation	Used only for anchor scoring, now obsolete

Joint SetB selection score (Av_ext - w*robustness)	Did not improve success rate; reverted to extinction-only
categorizeAndAnchor_v2 joint optimization	Hard to debug; replaced by sequential categorize + shuffle
Per-group shift mechanism in SetA (unrestricted)	Replaced by "at most one group shifts" rescue
Cross-set scoring (feas_tier*1e6 + ...)	Replaced by binary success/failure plus shuffle
Standalone correct_Inds with multi-steal bug	Replaced by matchpairs-based pre-clean + SetA-preserving Case B
Various commented-out v2 dev attempts	Cleaned

14. Helper functions (outside classdef)

Helper	Used by	Purpose
local_make_field_table	categorizeFields_v3	Build a standard 5-column field table from a list of field IDs
local_match_setA	schedule_SetA_v3	Bipartite max-cardinality matching: slot indices vs field row indices
local_setC_place_constrained_first	schedule_SetC_v3	Place SetC fields ascending by max_window, load-balanced across super-windows
local_setB_division	schedule_SetB_v3	Return W45/W90_1/W90_2 vectors for SetC_start_ind = 3 or 1
local_compute_slot_occupancy	clean_inds_before_setD	Compute inds_open / inds_2move from Schedule (cadence-aware, /4 check)
local_apply_setA_moves	clean_inds_before_setD	Apply (Field, from_ind, to_ind) moves to SetA rows; assigns new group
local_assign_moved_setA_group	local_apply_setA_moves / Case B	Lowest group ≥ 7 at target ind that is not already taken (7, 8, 9, ...)
local_commit_setD	schedule_SetD_v3	Append a SetD row to Schedule (category="D", group=300+slot)

15. Next steps

The full A/B/C/D/Daily pipeline is now in v3. Remaining items:

- Investigate the Case-A-first greedy SetD heuristic: replace with a global matchpairs over (4 placement slots) x (rank x ind feasibilities) to close the small gap between ABC and ALL success rates.
- Confirm SetA_shifted_group bookkeeping covers all edge cases (e.g. the orchestrator's shuffle-then-retry sequence triggering different shifts in different attempts).
- Consider adding a per-StartDate diagnostic table for sweep results (success/failure tier, which sci won, how many shuffles, which SetD ranks fired).
- Decide whether to expand the A/B/C shuffle strategy beyond the current option (a) primary + (b) fallback if the success rate is insufficient for the science goals.